Accès aux données et infrastructure Spring (cours 1)

F. Vargas-Rojas, I. Mougenot

Faculté des Sciences Université Montpellier

2026

Plan du premier cours

Mobiliser des données rendues permanentes au sein de Bases de Données Relationnelles (BDR)

1. framework Spring (Solution Spring Boot) et développement d'applications d'entreprise
2. focus sur la couche de persistance au sein d'une architecture multi-niveaux (multi-tier)
   - mobiliser les bonnes pratiques pour assurer le passage entre le relationnel et l'objet (et se dégager de la difficulté posée par la distorsion des langages)

UNIVERSITÉ DE MONTPELLIER

Quelques définitions préalables

1. Framework : Ensemble de classes abstraites et concrètes qui composent un modèle d'application. Mise en synergie de classes pour fournir un comportement par défaut
   Des Exemples de frameworks : Jakarta EE (anciennemen Java EE), .NET, Spring / Spring Boot, Struts, Micronaut.
2. Serveur d'application : logiciel d'infrastructure qui offre un contexte d'exécution (runtime) pour des composants applicatifs
   Par exemple Tomcat dans le contexte du framework Spring Boot

Application d'entreprise

solution logicielle conçue pour améliorer l'efficacité des processus métiers au sein des entreprises/organisations de grande taille (centaines voire milliers de collaborateurs implantés sur différents sites) et in fine la productivité de ces mêmes organisations

- les usages et besoins courants d'une entreprise : gestion de contenus et mémoire d'entreprise, commerce électronique et ventes, facturation et paye, gestion de la relation client, planification des ressources d'entreprise (ERP) (finances, achats, RH, fabrication, ...), business intelligence et analyse décisionnelle, chaînes d'approvisionnement et logistique, ...
- les attendus côté considération informatique : gestion centralisée, flexibilité, adaptabilité, sécurité, intégration entre autres

UNIVERSITÉ DE MONTPELLIER

Les fondements : plateforme Java Entreprise et ses évolutions

différentes acceptions de J2EE en passant par Java EE puis Jakarta EE depuis 2017 et la cession par Oracle du projet à la fondation Eclipse

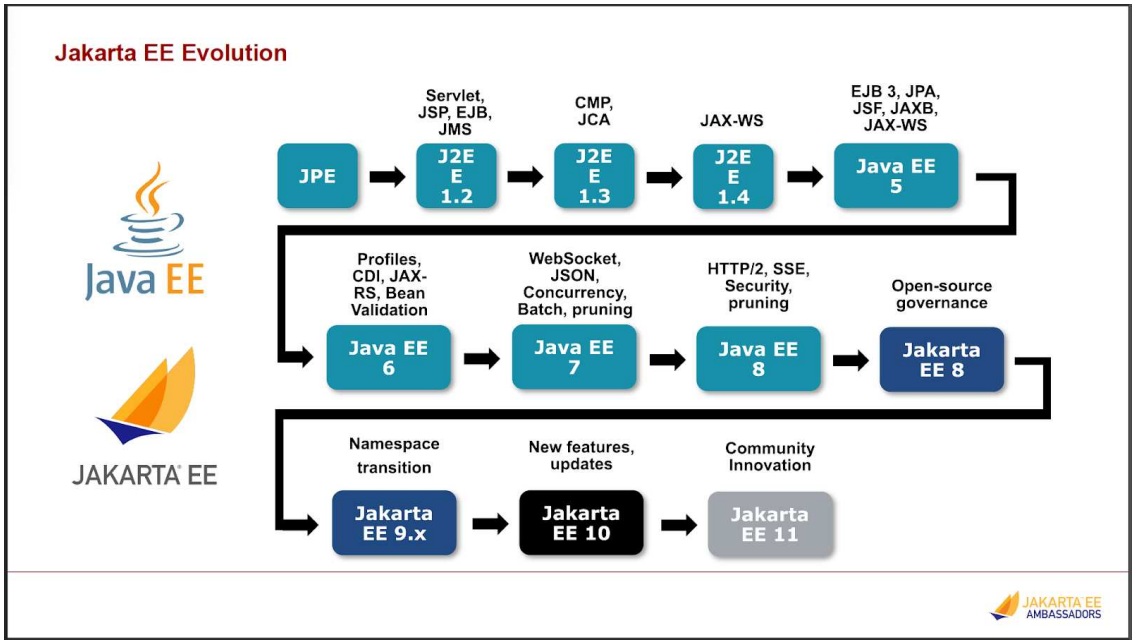


Figure: Evolutions du projet dans le temps

Un framework Java

Faciliter/Simplifier le développement d'applications d'entreprise

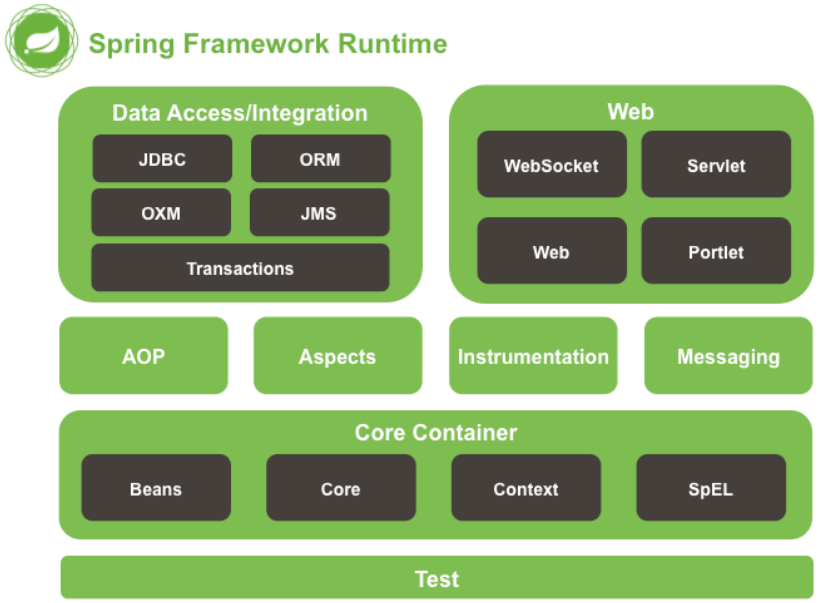


Figure: Vision modulaire (extrait de la documentation `https://docs.spring.io/spring-framework/docs`)

Le programmeur doit pouvoir ne se concentrer que sur la logique métier

Les points d'importance

- basé composants (forte cohésion et faible couplage)
- logique métier (rôle central joué par les POJOs (fondements des beans Spring) en lieu et place des EJBs (devenus obsolètes)
- mise en œuvre de la modularité via des patrons de conception (ici design patterns DAO et Repository)
- architecture multi-couches possiblement distribuée

Notion de composant

- un composant est un agrégat de classes qui répondent collectivement à un besoin fonctionnel : cohésion forte au sens métier
- composant = unité fonctionnelle qui pourra par ailleurs être évaluée au travers de tests unitaires
- faible couplage des composants pour gagner en modularité au niveau de l'application
- comportement d'un composant défini en terme d'interfaces fournies et requises

UNIVERSITÉ DE MONTPELLIER

Mise en œuvre : se "simplifier" la tâche

Réduire le temps de montée en compétence du développeur

- Spring Initializr
- Spring Boot : automatisation de la configuration : le principe par défaut de convention
  - CLI (Command Line Interface) pour faciliter les tests et prises en main depuis la console
  - serveur d'application intégré sans nécessité de configurations supplémentaires (TomCat ou Jetty)
  - plugins pour utiliser directement des BD embarquées (H2)

UNIVERSITÉ DE MONTPELLIER

Description de l'architecture

Trois grands modes possibles

- par fichier de configuration XML (les "beans" y sont décrits)
- par programmation
- **par annotations Java (fournies par le framework) au sein même du "bean"** -> choix retenu ici

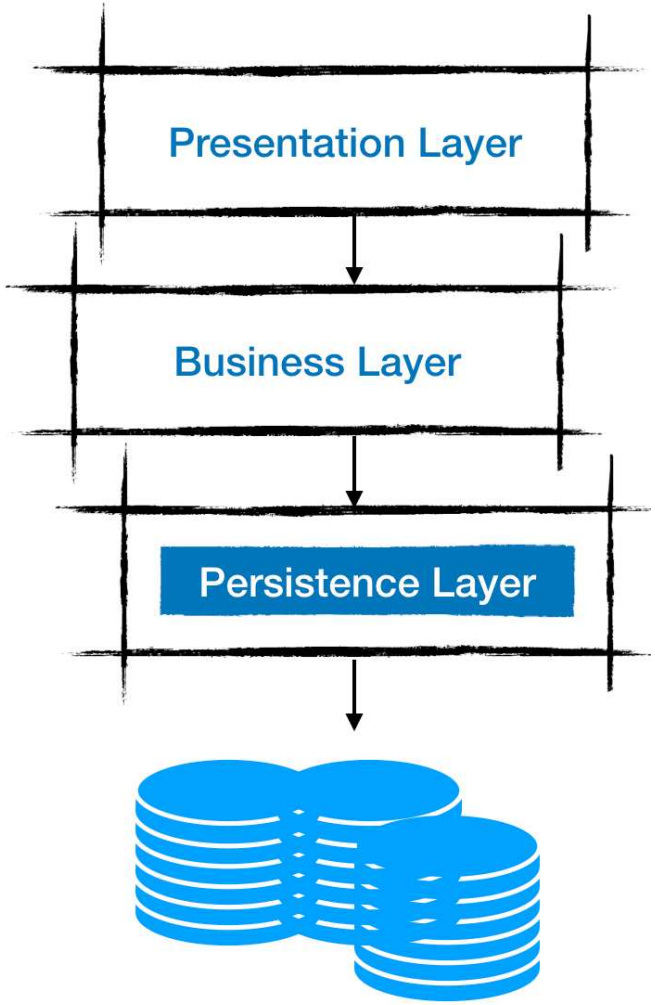
Exemples d'annotation/décorateur

- @Autowired (auto-connecter un bean - auto-connexion et auto-découverte du bean par le framework)
- @Component - composant générique
- @Repository - composant spécialisé accès aux données (DAO) (opérations persistance)
- @Service - composant spécialisé Service (logique métier)
- @Controller - composant spécialisé orienté web (SPRING MVC et prise en charge requête HTTP)
- @Bean - sorte de moule pour la définition d'objets
- @Configuration : à la source de définition de "beans"

UNIVERSITÉ DE MONTPELLIER

Architecture multi-couches (multi-tier)

Focus sur la couche de persistance (médiation de données)



Règles de passage entre la source de données et l'application

Diverses façons d'envisager le passage entre la source de données externe (pas obligatoirement une BD relationnelle) et la POO, avec à la clé 3 patterns :

1. Pattern "orientée données" (Data Oriented) voir Livre "Data-Oriented" de Yehonathan Sharvit
2. Pattern "enregistrement actif" (Active Record) voir Livre "Patterns of Enterprise Application Architecture" de Martin Fowler
3. Pattern "mise en correspondance" (Mapper) : séparation claire entre les opérations sur l'entité métier et les opérations (CRUD) sur la BD voir de nombreux livres sur le sujet à l'exemple de "Core J2EE Patterns: Best Practices and Design Strategies" de Deepak Alur, John Crupi, et Dan Malks

UNIVERSITÉ DE MONTPELLIER

Solution de mapping (la seule étudiée ici)

Deux patrons de conception d'importance :

1. Pattern DAO (Data Access Object) : abstraction sur les opérations CRUD (Create, Retrieve, Update, Delete)
2. Pattern Repository (Entrepôt) : encapsule les comportement de stockage, et de recherche et de restitution sur la base d'une collection d'objets
3. Le pattern Repository (focus sur la collection d'objets) peut s'utiliser de concert avec le pattern DAO (focus sur l'objet)

Regard uniquement centré sur le mapping objet/relationnel (appelé ORM)

Relationnel et programmation externe

1. la distorsion des langages (impedance mismatch) : les LDD et LMD (déclaratifs et orientés ensemble) versus les langages de programmation *dit hôtes* (impératifs et orientés élements) présentent des incompatibilités qui nécessitent des artifices de programmation (curseurs par exemple).
2. Les langages de programmation sont souvent plus complets en ce qui concerne la prise en charge de la complexité des objets du monde considéré.

Difficultés de mise en place de couches applicatives implantées autour des SGBD relationnels

UNIVERSITÉ DE MONTPELLIER

Patron "Data Access Object"

Orienté vers la persistance des objets : un ensemble de classes intermédiaires (suffixées par DAO) se placent entre la couche métier et la base de données. Les classes DTO (objets de transfert de données) sont facultatives

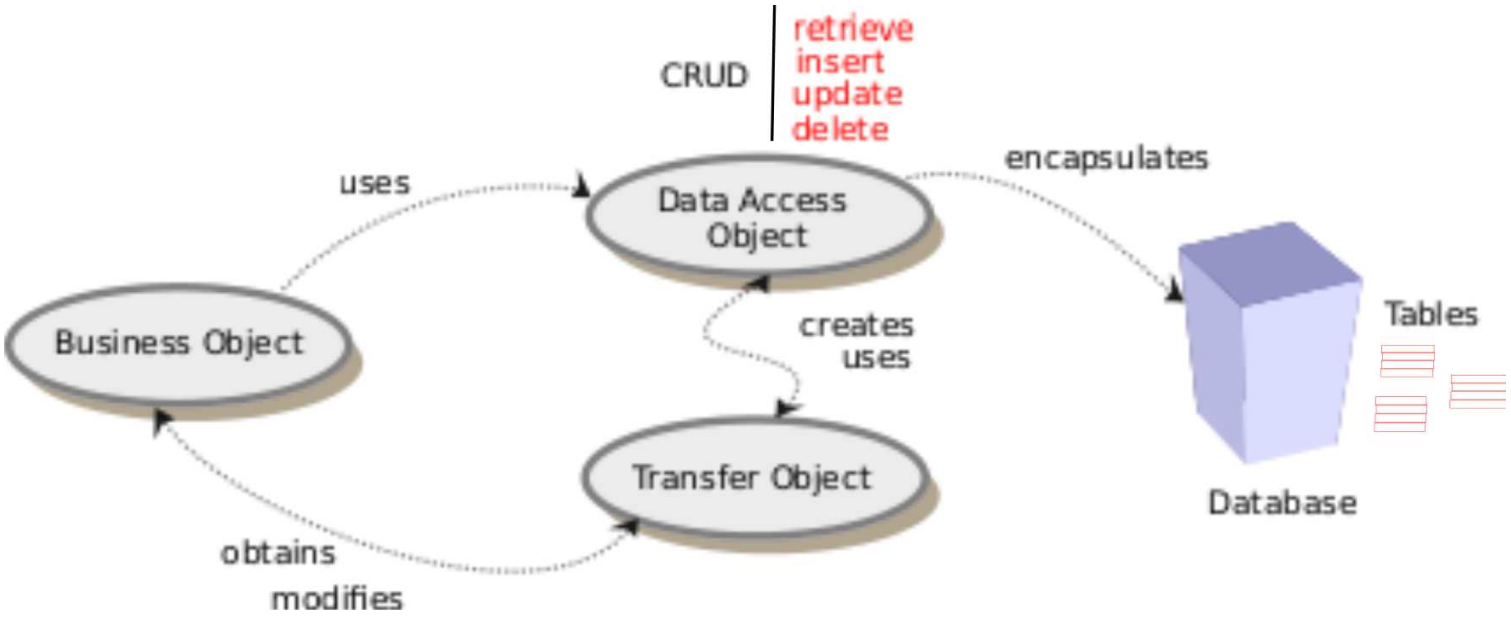


Figure: Patron de conception DAO

Design Pattern DAO (pour Data Access Object)

Les classes *DAO* vont prendre en charge les opérations de consultation, d'insertion et de mise à jour permises sur les tuples des tables de la bd (acronyme CRUD)

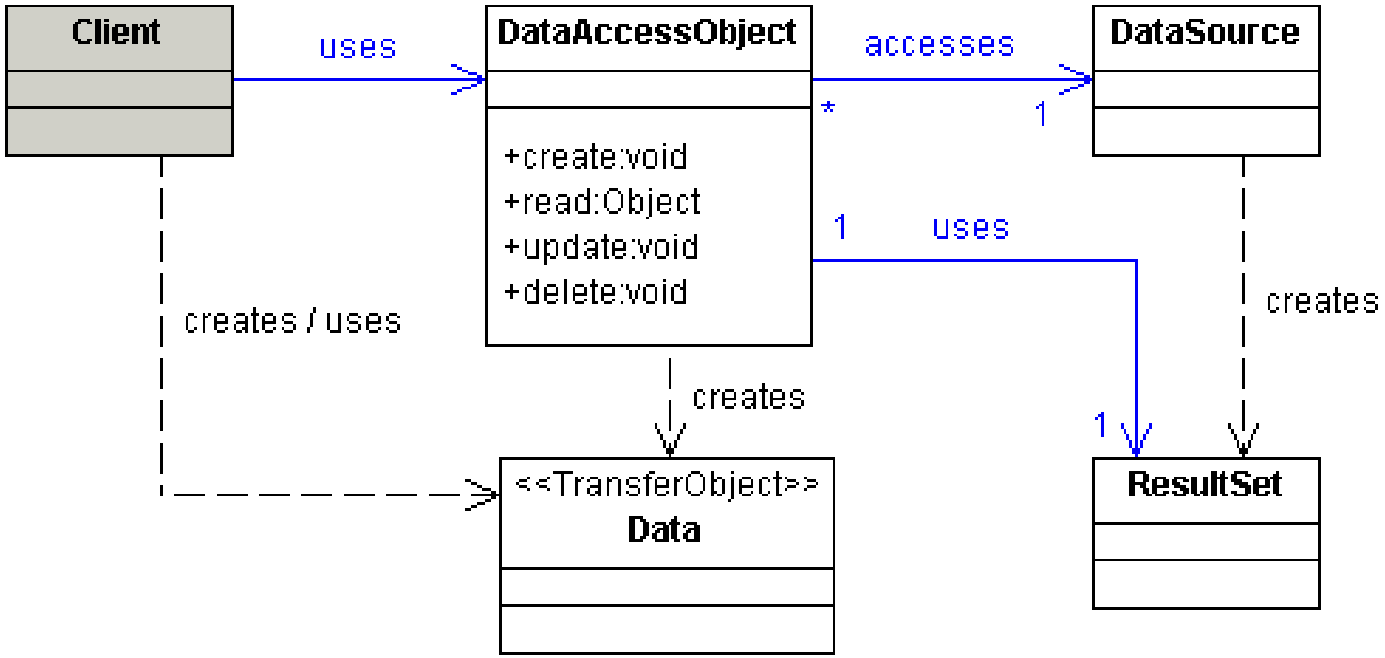


Figure: Patron de conception DAO au travers d'un diagramme UML simplifié

Patron Data Access Object

Mise en œuvre du pattern DAO

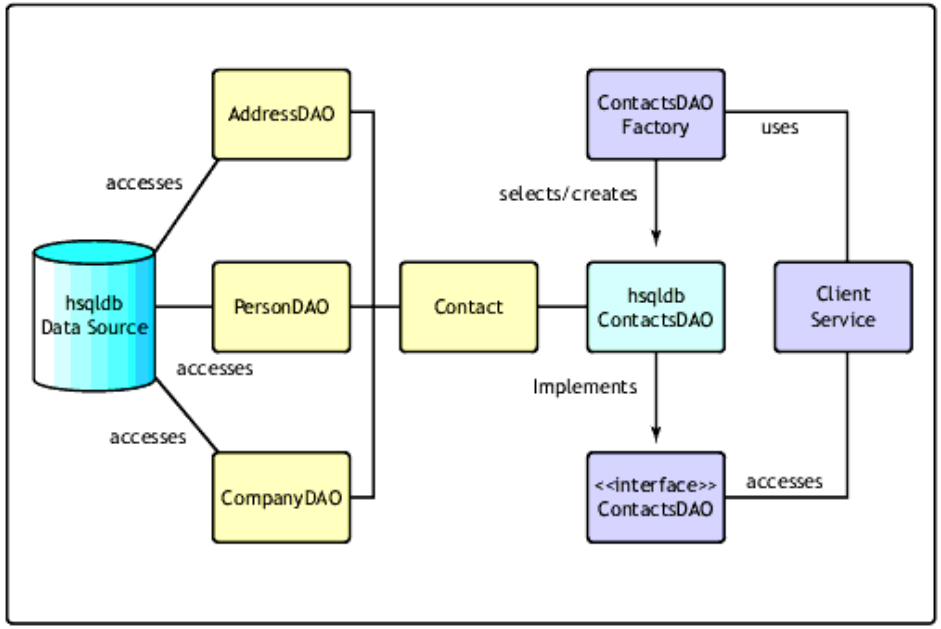


Figure: Contexte d'un SGBD spécifique qui peut s'étendre à tout type de SGBD

Patron Repository

Ce patron centré sur le domaine sous-jacent à l'application va faciliter maintenabilité, séparation des préoccupations, et outiller l'inversion de contrôle

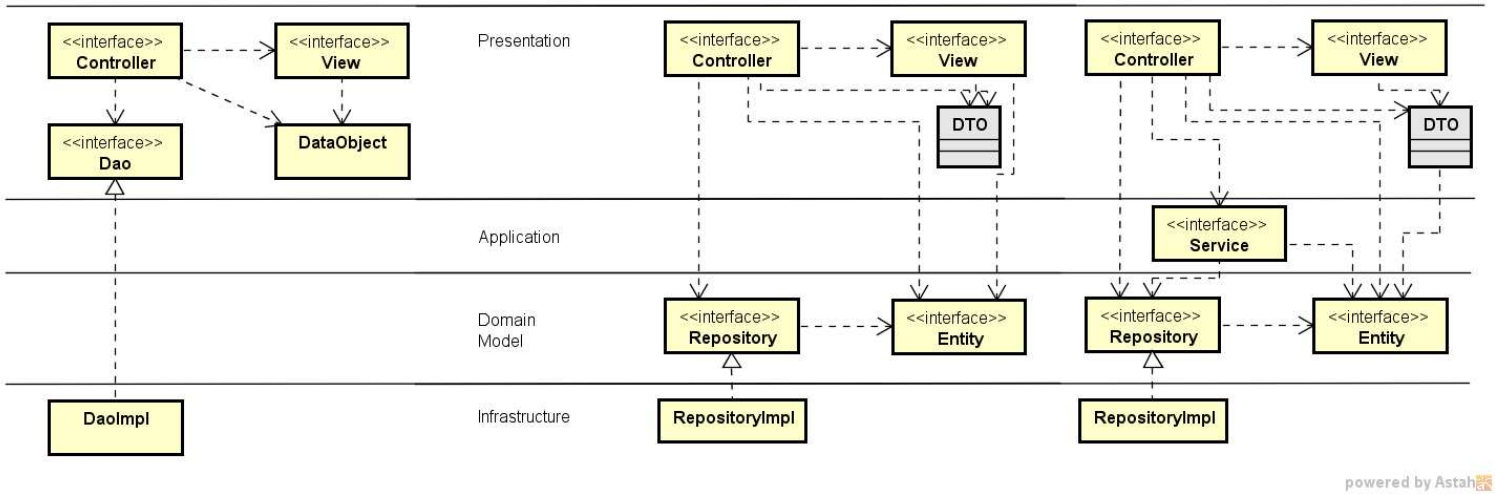


Figure: Diagramme de classes illustrant l'intérêt du patron Repository

Couche de persistance : fonctionnalités attendues

Des fonctionnalités élémentaires :

1. Gérer objets et collections d'objets
2. Persister/Mettre à jour les objets avec exécutions des instuctions CRUD (Create, Retrieve, Update et Delete)
3. Gérer les relations entre entités (associations et leurs multiplicités, héritage, agrégation)

Des fonctionnalités avancées :

1. sérialisation et un déploiement physique multi-tier
2. cache pour optimisation des performances
3. chargement tardif (lazy loading)
4. traçabilité des actions sur la base de données (journalisation)

UNIVERSITÉ DE MONTPELLIER

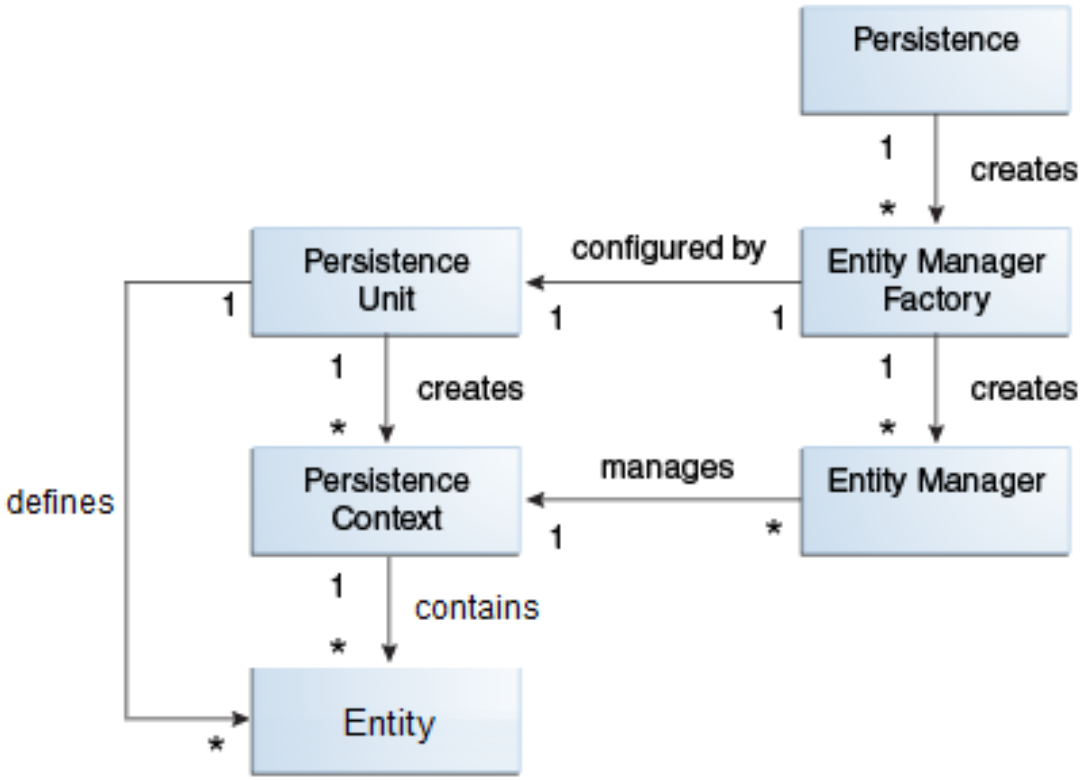
Pour l'accès aux données dans SPRING

Plusieurs modules disponibles

- SPRING JDBC
- **SPRING DATA JPA** (besoin d'une solution JPA, par exemple HIBERNATE)
- SPRING DATA REST
- SPRING DATA NEO4J
- SPRING DATA MONGODB
- ...

Standard JPA

Spécification standard pour la persistance des objets en Java. Depuis 2017 le paquetage javax.persistence est devenu jakarta.persistence
Parmi les implémentations de JPA les plus usuelles : Hibernate, OpenJPA, EclipseLink ou DataNucleus



Solution SPRING DATA JPA

Ensemble de modules applicatifs donnant lieu à une implémentation de "Repository" pour une catégorie de SGBDs

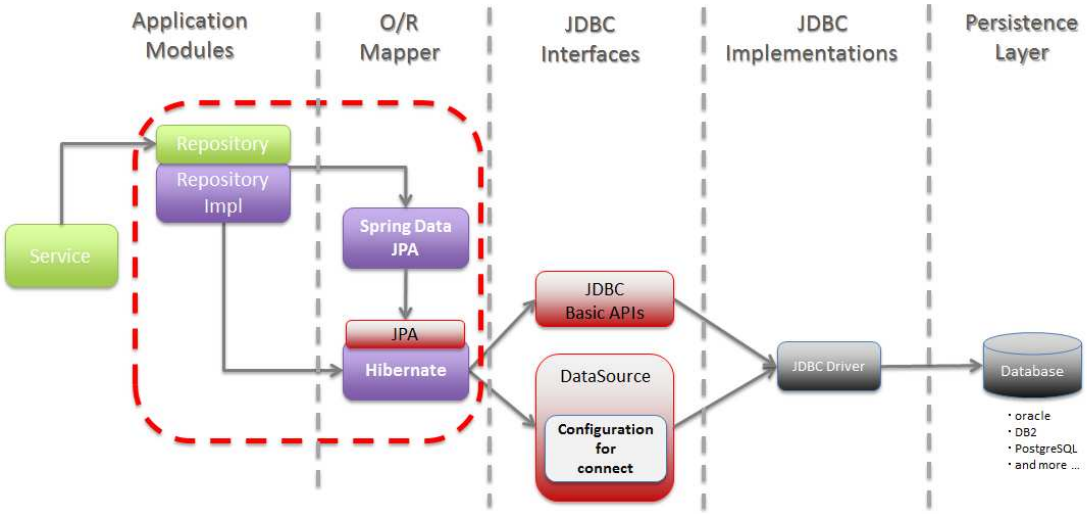


Figure: Articulation possible pour SPRING DATA JPA

SPRING DATA JPA

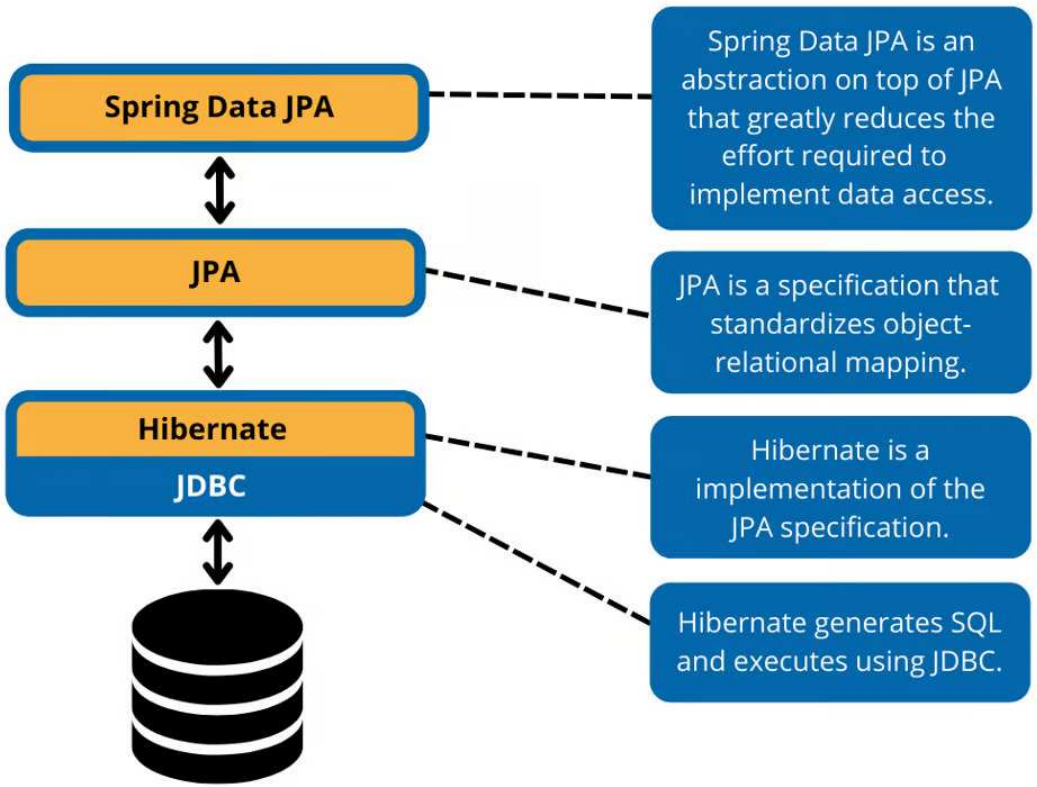


Figure: Interfaces d'accès aux sources de données extrait de `https://codingnomads.com/spring-data-jpa`

Grandes notions autour de JPA

Standard Javax Persistence API / désormais Jakarta Persistence API

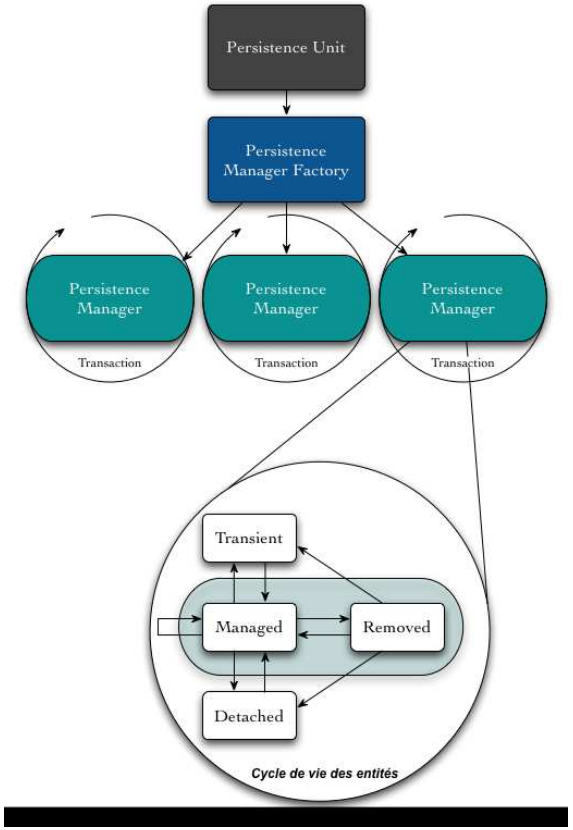


Figure: Entités régies par un gestionnaire dans un contexte de persistance

Cycle de vie des entités

Les différents états possibles d'une entité

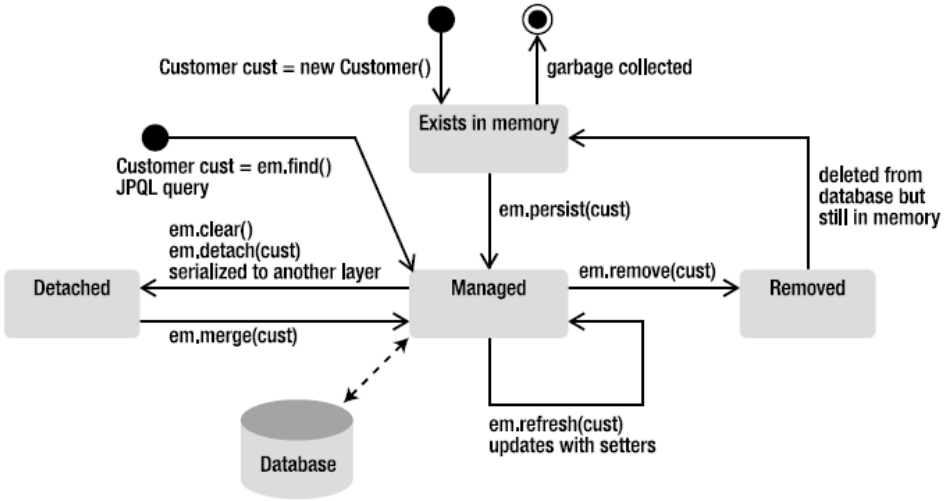


Figure: Diagramme états-transitions UML

Cycle de vie des entités

Les quatre états principaux et la prise en charge par le contexte de persistance (noté CP)

New/Transient	l'entité est créée	hors du CP
Managed/Persistent	l'entité est "persistée" (après création ou retournée par la BD)	dans CP
Detached	l'entité est sortie du CP	hors du CP
Removed	l'entité est étiquetée comme étant à supprimer	hors du CP

Cycle de vie des entités

Autre vision s'appliquant au cycle de vie (4 cas de figure)

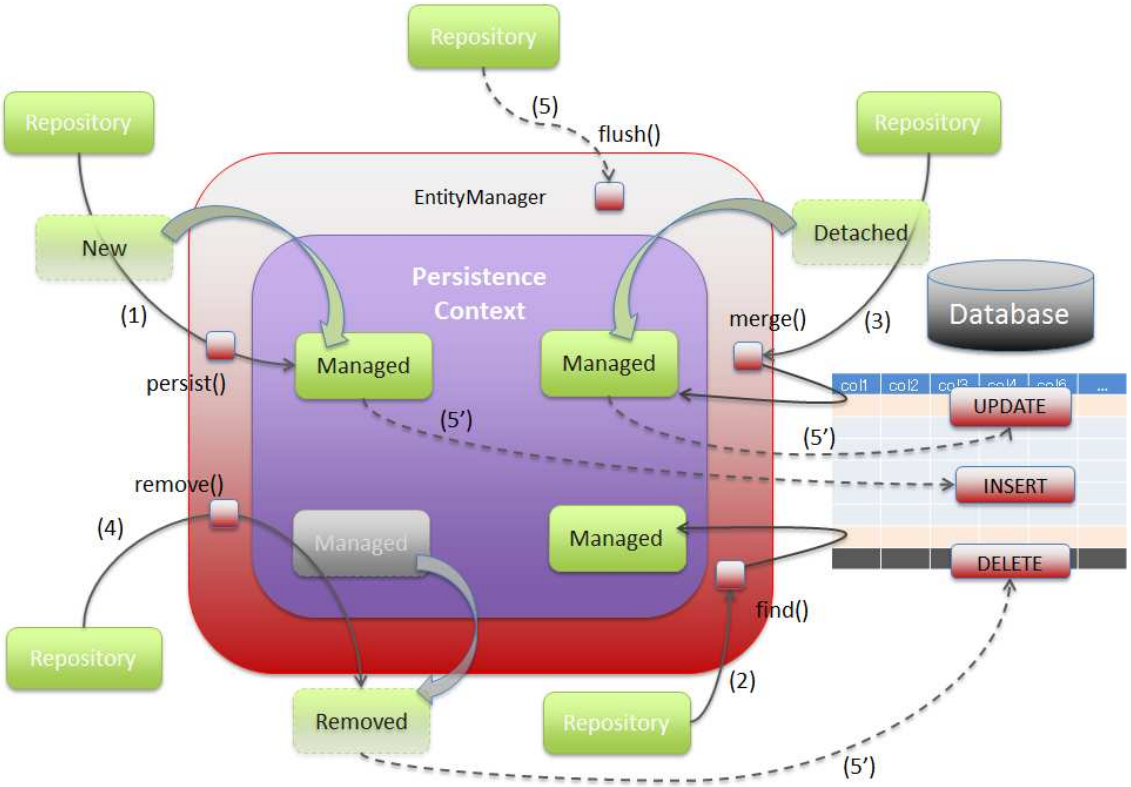


Figure: Vision intégrée sur les entités

Un exemple repris en TP

Diagramme de classes descriptives ou POJO
POJO : Plain Old Java Object : le reflet d'objets élémentaires qui sont les entités ou les objets du domaine de l'application (sans comportement autre que de la manipulation avec des constructeurs et accesseurs). Ces objets à l'origine ne sont pas dépendants de librairies externes ou de frameworks

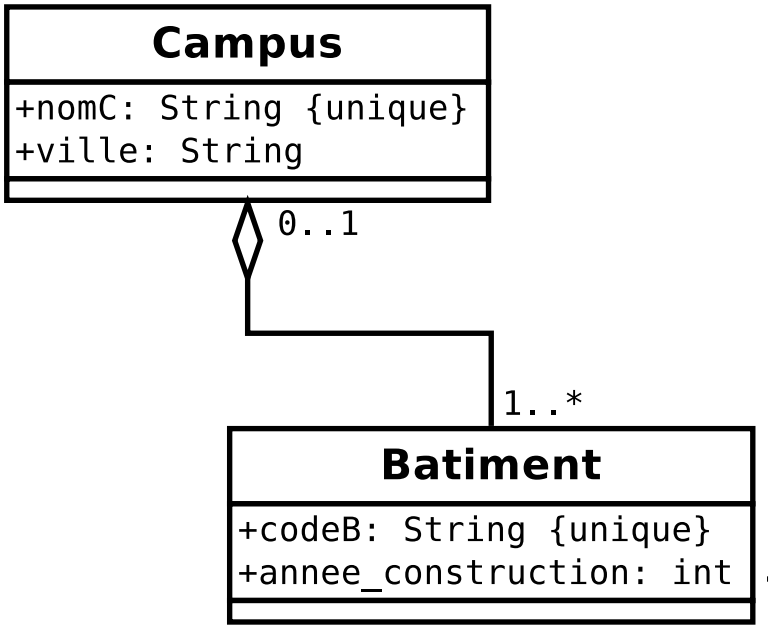


Figure: Diagramme de classes du modèle exemple

La plus grosse difficulté : information provenant de la recomposition de tables

- campus (**nomC varchar(16)**, ville varchar(20))
- batiment (**codeB varchar(16)**, anneeC integer, #campus varchar(16));

référence sur la valeur de l'attribut clé versus référence sur l'objet. Ici une valeur textuelle pour campus contre un objet de campus

Classe POJO Batiment : identifier la difficulté

```
public class Batiment {
   private String codeB;
   private int anneeC;
   private Campus campus;

   public Batiment() {}
   public Batiment(String codeB, int anneeC, Campus campus) {
      this.codeB = codeB;
      this.anneeC = anneeC;
      this.campus = campus; }

   public Campus getCampus() {
      return campus;
   }
   public void setCampus(Campus campus) {
      this.campus = campus;
   }
}
```

Listing 1: Extrait POJO

Entity au sens JPA

JPA Entity : un POJO mis en correspondance avec une ou plusieurs tables d'une base de données relationnelle

```
import jakarta.persistence.*;

@Entity
@Table(name="batiment")
public class Batiment {
   @Id
   @Column(name="codeB", length = 16)
   private String codeB;
   private int anneeC;
   @ManyToOne(optional = true)
   @JoinColumn(name = "campus")
   private Campus campus;

   public Batiment() {}
}
```

Listing 2: Exemple partiel avec les annotations

Annotations portant sur une entité JPA

```
@Entity import jakarta.persistence.Entity;
@Table import jakarta.persistence.Table;
@Column import jakarta.persistence.Column;
@Id import jakarta.persistence.Id;
@GeneratedValue import jakarta.persistence.GeneratedValue;
@Version import jakarta.persistence.Version;
@OrderBy import jakarta.persistence.OrderBy;
@Transient import jakarta.persistence.Transient;
@Lob  import jakarta.persistence.Lob;
```

Listing 3: Exemples d'annotations (non exhaustif)

Annotations : Pour guider les "mappings" (très partiel)

@JoinColumn	indique la colonne de jointure correspondant à une entité associée
@ManyToMany	marque une association plusieurs à plusieurs entre entités
@ManyToOne	marque une association plusieurs à un entre entités
@OneToMany	marque une association un à plusieurs entre entités
@OneToOne	marque une association un à un entre entités
@JoinTable	indique la table support du mapping plusieurs à plusieurs ou un à plusieurs
@Inheritance	indique la stratégie associée pour une mise en correspondance d'une hiérarchie de classes

voir

https://javadoc.io/doc/jakarta.persistence/jakarta.persistence-api/latest/jakarta.persistence

Mapping bidirectionnel de Campus vers Batiment

```
@Entity
public class Campus {

   @Id
   private String nomC;
   private String ville;

   @OneToMany(fetch = FetchType.LAZY, mappedBy="campus",
       cascade = CascadeType.REMOVE)
   private List<Batiment> batiments = new ArrayList<>();

   public Campus() {}
}
```

Listing 4: inverse de ManyToOne de Batiment

Illustration mapping bidirectionnel

Avec coupure de l'association et migration des attributs de part et d'autre

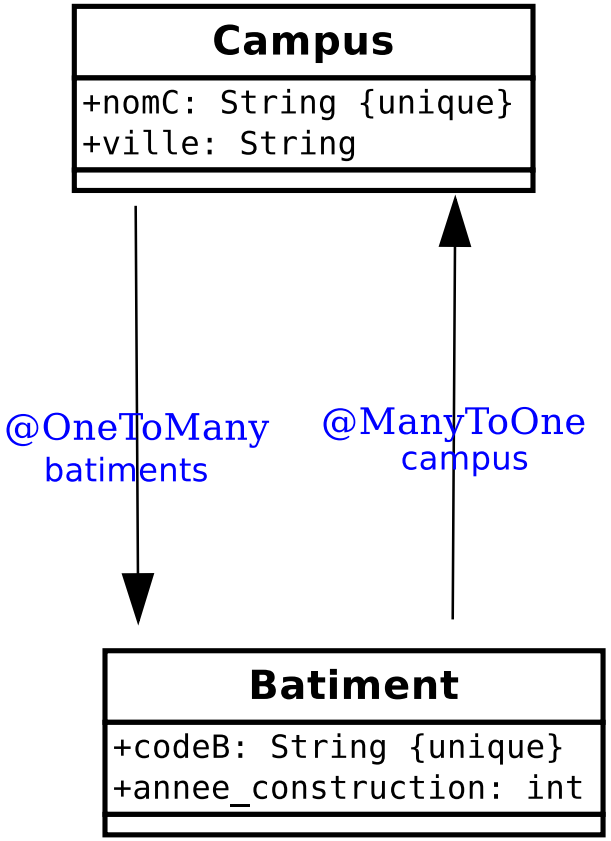


Figure: Mappings

Composant JPA Repository

Appel à deux classes génériques : ici T qui correspond au type de l'entité sous la responsabilité de maintenance par le repository et ID qui correspond au type de l'attribut identifiant de l'entité

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;;

@Repository
public interface JpaRepository<T, ID> {
}
```

Listing 5: Exemple de l'interface Repository pour Batiment

Illustration du composant JPA Repository

Interface qui encapsule la logique des accès aux BD (opérations CRUD). Ici l'entité est Batiment et l'identifiant CodeB est une chaîne de caractères

```
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;;

@Repository
public interface BatimentRepository extends
    JpaRepository<Batiment, String> {

}
```

Listing 6: Exemple de l'interface Repository pour Batiment

Jpa Repository : étend plusieurs interfaces

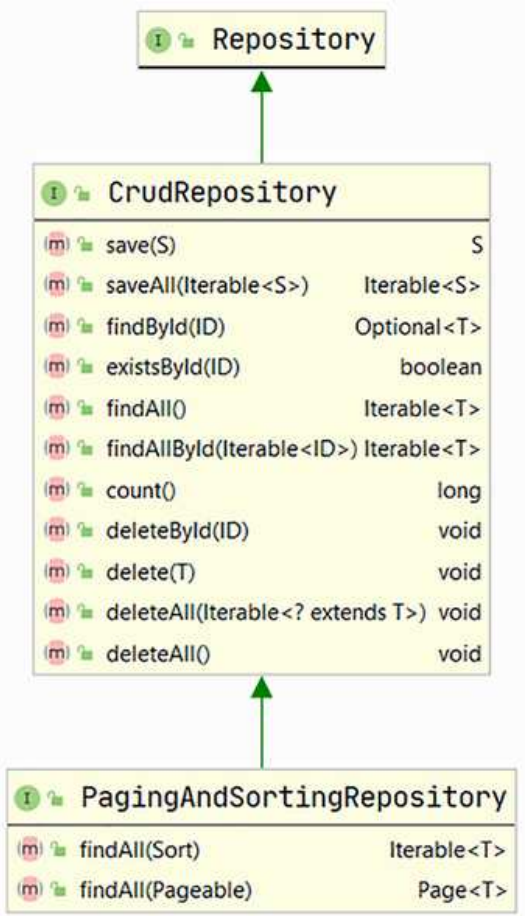


Figure: Repository, CrudRepository, PagingAndSortingRepository

Repository : un des composants qui vient spécialiser Spring Component

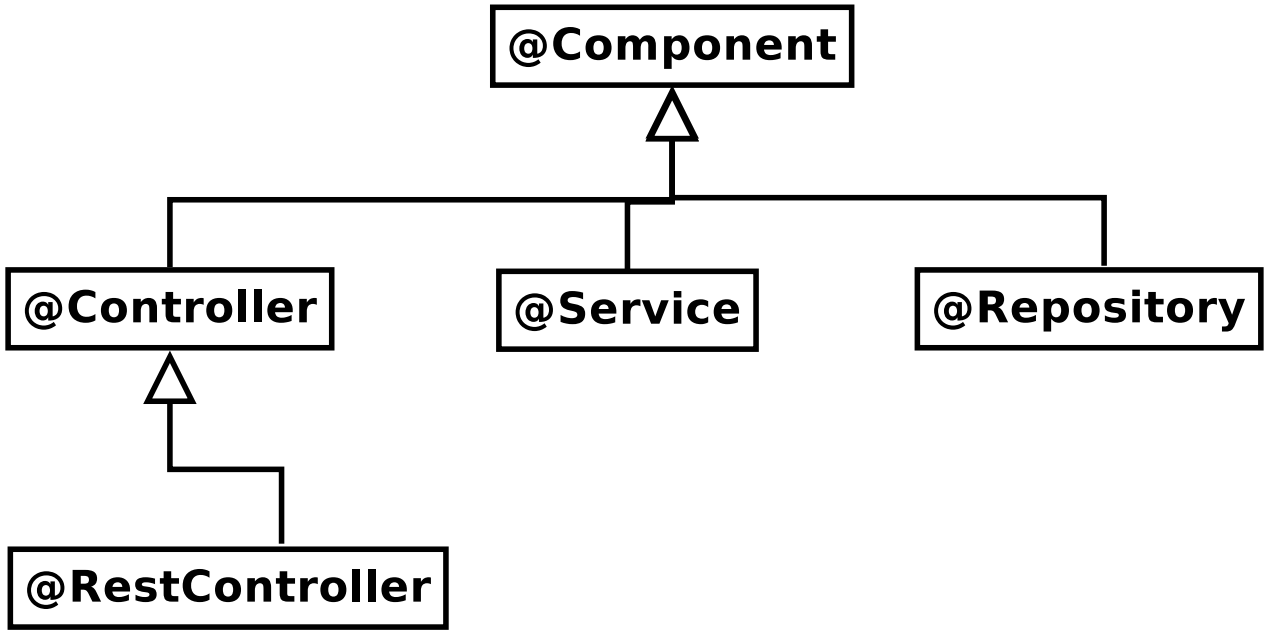


Figure: Principaux composants de Spring

Repository : siège de l'injection de dépendances

Les objets DAO sont injectés dans les services qui séparent la couche métier de la couche de persistance, et sont utilisés pour communiquer avec la BD

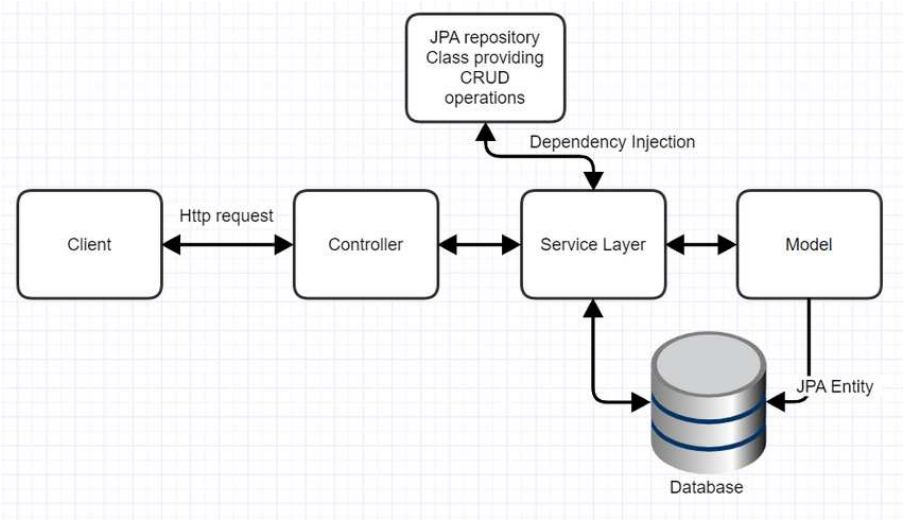


Figure: Articulation des composants au sein d'une application Spring

Mobiliser les langages de requêtage

Les interfaces Repository à étendre sont riches de méthodes, il est cependant possible de définir des requêtes à l'aide des langages JPQL et SQL

- requêtes SQL dites natives (SQL étant le standard pour les BDR) nommées ou non nommées
- requêtes JPQL nommées ou non nommées

UNIVERSITÉ DE MONTPELLIER

Exemple de requête SQL au sein de l'interface SalleRepository

```
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

@Repository
public interface SalleRepository extends JpaRepository<Salle,
    String> {

@Query(value = "SELECT * FROM salle ORDER BY batiment",
    nativeQuery = true)
List<Salle> findAllOrderByBatiment();

}
```

Listing 7: SalleRepository

Le langage JPQL

JPQL ou Java Persistence Query Language combine syntaxe du SQL avec des caractéristiques à rapprocher d'un langage d'expression orienté objet

- JPQL est une extension du langage EJB QL (Enterprise JavaBeans Query Language), avec EJB ayant eu le standard de requêtage objet OQL (Object Query Language) comme source d'inspiration. OQL est avec ODL (Object Definition Language) les standards proposés par l'ODMG (Object Data Management Group) pour couvrir les besoins des SGBDOO (à l'exemple de O2, Matisse, EyeDB ou Db4O)
- HQL (Hibernate Query Language), mobilisable dans les implémentations basées sur Hibernate, est un sur-ensemble de JPQL

Exemples de requêtes JPQL usuelles

Opérer des projections, sélections et jointures

```
SELECT c FROM campus c

SELECT s FROM Salle s WHERE s.typeS = 'numerique'

SELECT c.nomC, bts.codeB FROM Campus c JOIN c.batiments bts

SELECT c.nomC, bts.codeB, sls.numS FROM Campus c JOIN
    c.batiments bts join bts.salles sls
```

Listing 8: SELECT/FROM/WHERE

Requête JPQL au sein de l'interface CampusRepository

```
@Repository
public interface CampusRepository extends
    JpaRepository<Campus, String> {

   @Query("SELECT c.nomC, bts.codeB, sls.numS FROM Campus c
       JOIN c.batiments bts join bts.salles sls")
   List<String[]> campusEtCodeBEtNumS();

}
```

Listing 9: Corps CampusRepository

Traduction de la requête par le framework Hibernate ORM

```
select c1_0.nomc,b1_0.codeb,s1_0.nums
from campus c1_0 join batiment b1_0
on c1_0.nomc=b1_0.campus join salle s1_0 on
    b1_0.codeb=s1_0.batiment
```

Listing 10: Traduction en SQL

Appel à la méthode campusEtCodeBEtNumS dans le code d'exécution

```
public void campusEtBatimentsEtSalles()
   {
   Iterable<String[]> e = cr.campusEtCodeBEtNumS();
   System.out.println("campus qui ont des batiments (codes)
       qui ont des salles (numS)");
       if (StreamSupport.stream(e.spliterator(),
           false).count() != 0) {
     for (String[] s : e)
       { System.out.println(" ");
       for (String entry : s) {
           System.out.print(entry+" ");
         }
       }
     } else System.out.println("BD vide ?");
   }
```

Listing 11: Appel de la méthode (cr : objet CampusRepository)